

Integration Testing Strategy

Caseware Collaborate — Authorization Layer

Goal: Prove that the L1/L2 epoch-based revocation system and OBO delegation policy work end-to-end, using a *real Redis container*, the *real ASP.NET Core pipeline*, and *real JWTs* — no mocks in the critical path.

1. Testing Pyramid



Layer	Scope	Tools	Speed
Unit	Single class, all dependencies mocked	xUnit, Moq/NSubstitute	< 1 s
Integration	Full HTTP pipeline, real Redis	xUnit, WebApplicationFactory, Testcontainers	10-30 s
E2E	Deployed service, real IdP	Postman, k6	Minutes

Integration tests own the boundary contracts. They answer the question that unit tests cannot: *does the authorization middleware, Redis epoch store, and OBO delegation policy work correctly as a composed system?*

2. What WebApplicationFactory<Program> Gives Us

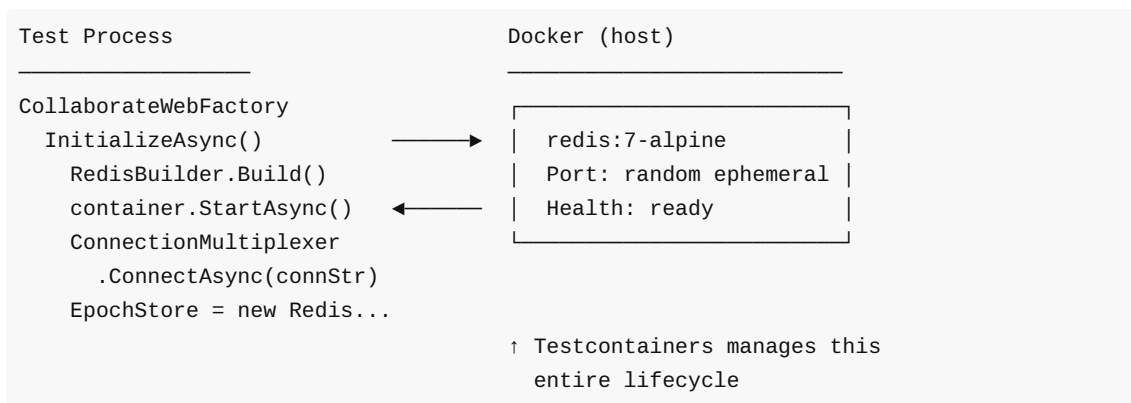
`WebApplicationFactory<T>` boots the **exact same** ASP.NET Core pipeline that runs in production — same middleware order, same DI container, same `appsettings.json` — but entirely in-process. No network, no Docker for the API itself.

This means:

- `JWT Bearer` middleware validates tokens with the **real** `JwtSecurityTokenHandler` .
- `OboDelegationHandler` runs the **real** epoch check against the **real** Redis.
- The endpoint extracts and returns claims from the **real** `ClaimsPrincipal` .

The only thing we replace (via `ConfigureTestServices`) is the `IPermissionEpochStore` registration — swapping the `NullPermissionEpochStore` for a `RedisPermissionEpochStore` pointing at the test container.

3. Testcontainers — Real Redis, Zero Setup



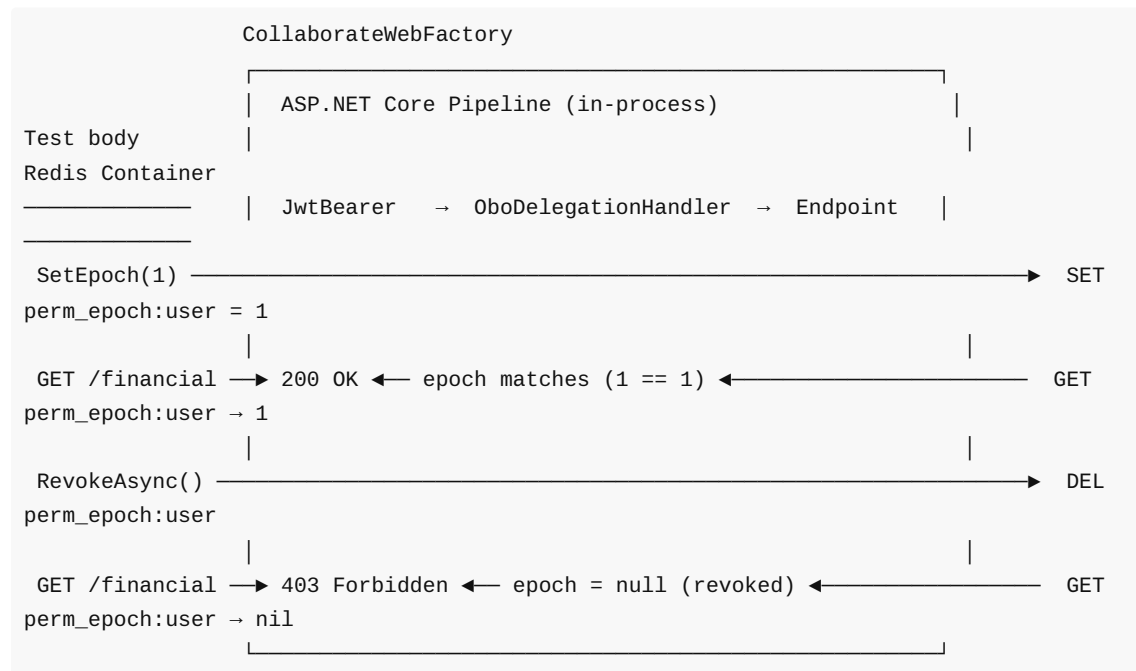
Why `redis:7-alpine` ?

- Minimal image (~25 MB) — fast pull in CI.
- Deterministic version pin — no surprise API changes.
- Testcontainers waits for the health check before returning, so there is no timing race.

Lifecycle:

1. `InitializeAsync` → container starts before any test in the class.
2. Tests run (one container instance shared across the class).
3. `DisposeAsync` → container is stopped and removed, even on test failure.

4. The Core Revocation Test — Step by Step



Why the second request returns 403

The JWT has **not expired** — its signature is still valid and its `exp` claim is in the future. What changed is the Redis key. `OboDelegationHandler.HandleRequirementAsync` calls:

```
var storedEpoch = await epochStore.GetCurrentEpochAsync(userId);

if (storedEpoch is null || storedEpoch.Value != jwtEpoch)
    return; // requirement not satisfied → 403
```

`storedEpoch` is now `null` (key deleted). The handler falls through without calling `context.Succeed()`. ASP.NET Core's authorization middleware sees an unsatisfied requirement and returns **403 Forbidden**.

This is **sub-second revocation** — the Redis `DEL` propagates instantaneously. The only remaining stale window is the L1 in-memory cache TTL (≤ 2 s), which is acceptable per the ADR.

5. Test Suite Overview

5a. `WhenPermissionsRevoked_SubsequentRequest_Returns403` (*core scenario*)

```
Seed Redis: epoch=1
Mint JWT: sub=user-001, perm_epoch=1, act=svc:reporting, scope=financial:read

→ GET /api/v1/workspaces/ws-42/financial-data [EXPECT 200]
  Assert body.subject == "user-integration-001"
  Assert body.delegated_by.sub == "svc:reporting-service"
  Assert body.scope_used == "financial:read"

→ RevokeAsync(userId) [simulate perm.revoked event]

→ GET /api/v1/workspaces/ws-42/financial-data [EXPECT 403]
  (same JWT, cryptographically valid, but epoch is gone)
```

5b. `WhenActClaimMissing_Request_Returns403` (*Confused Deputy protection*)

```
Mint JWT: sub=user-direct, scope=financial:read
(no act claim – simulates a user calling the downstream API directly)

→ GET /financial-data [EXPECT 403]
  (OboDelegationHandler Check 1 fails: act claim absent)
```

5c. `WhenScopeInsufficient_Request_Returns403` (*scope narrowing*)

```
Seed Redis: epoch=1
Mint JWT: sub=user-scope-test, act=svc:reporting, scope=documents:read ← wrong scope

→ GET /financial-data [EXPECT 403]
  (OboDelegationHandler Check 3 fails: financial:read not in granted scopes)
```

6. Project Structure

```
TokenExchangeApi.Tests/
├─ Caseware.Collaborate.Tests.csproj ← xunit, WebApplicationFactory,
Testcontainers.Redis
├─ CollaborateWebFactory.cs          ← IAsyncLifetime fixture; owns Docker
container + override
├─ JwtTestHelper.cs                 ← Centralized JWT factory; mirrors
TokenExchangeService output
└─ PermissionRevocationTests.cs     ← 3 integration tests; no mocks in the happy
path
```

7. Running the Tests

```
# Prerequisites: Docker must be running on the host machine.

cd TokenExchangeApi.Tests
dotnet test --configuration Release --logger "console;verbosity=detailed"
```

Expected output (first run — Docker pull):

```
Testcontainers: Pulling image redis:7-alpine ...
Testcontainers: Container started in 3.2 s

[PASS] WhenPermissionsRevoked_SubsequentRequest_Returns403
[PASS] WhenActClaimMissing_Request_Returns403
[PASS] WhenScopeInsufficient_Request_Returns403

Test Run Successful. Total: 3, Passed: 3 (~ 18 s including container startup)
```

In CI (GitHub Actions / Azure Pipelines):

```
- name: Run Integration Tests
  run: dotnet test TokenExchangeApi.Tests --configuration Release
      # Docker socket is available by default on ubuntu-latest runners.
      # No additional service containers needed – Testcontainers handles it.
```

8. Key Design Choices

Choice	Rationale
Real Redis, not a fake	Fakes cannot catch Redis key expiry behaviour, pipeline failures, or serialisation bugs. Testcontainers removes the setup cost with no trade-off.
One fixture per class	<code>IClassFixture<CollaborateWebFactory></code> starts Redis once for all tests in the class, keeping the suite fast.
Local DTO, not internal type	<code>ResponseBody</code> is defined in the test project. Tests should not depend on internal types of the system under test — changes to

	AuditableAccessResponse won't silently break test assertions.
ConfigureTestServices for DI override	Replaces only what needs replacing (IPermissionEpochStore). Everything else — auth middleware, endpoint registration, serialiser config — runs unmodified from Program.cs.
RequirePermissionEpochValidation flag	Decouples the Redis dependency from the dev inner loop. Tests activate it; appsettings.json leaves it off by default. Production appsettings.Production.json sets it to true.